

Git & GitHub를 통한 협업

체크리스트부터 고급 워크플로우까지

체크리스트

- ✓ 커밋 메시지 컨벤션(Commit Convention)의 필요성 및 기본 메시지 작성 템플릿 숙지 여부
- ✓ GitHub Issue의 개념적 역할 및 프로젝트 관리 도구(Jira, Projects 등)와의 연계성 이해 여부
- ✓ Issue Template 설정을 통한 일관성 있는 요구사항 및 버그 제보 포맷 구축 가능 여부
- ✓ GitHub Flow의 핵심 단계를 명확히 설명하고 로컬-원격 간 실무 워크플로우 적용 가능 여부
- ✓ Git Flow 및 GitLab Flow 브랜치 전략과의 개념적 장단점 비교 및 상황별 선택 가능 여부
- ✓ Pull Request(PR) 작성의 베스트 프랙티스(변경 내역 요약, 테스트 연동 등) 적용 가능 여부
- ✓ Collaborator 쓰기 권한 제한 방식과 Forking Workflow의 아키텍처적 차이점 분석 및 실행 여부
- ✓ Gemini Code Assist 등 AI 자동화 코드 리뷰 연동을 통한 개발 생산성 개선 원리 이해 여부

깃 메시지 컨벤션 (개요)

■ 필요성

- 히스토리 가독성 극대화
- 변경 이유 역추적 용이성 확보
- CHANGELOG(변경 이력 문서) 자동 생성 기반 마련

■ 표준 구조

타입(Type): 제목(Subject)

* 필요시 본문(Body) 및 꼬리말(Footer) 추가

깃 메시지 컨벤션 (핵심 태그)

feat

새로운 기능 구현 및 추가

fix

버그 현상 수정

docs

문서 파일 작성 및 갱신 (README.md, API 명세, 주석 등)

style

코드 스타일 수정 (포매팅, 세미콜론 누락 등 로직 영향 없음)

refactor

가독성 및 효율성을 위한 구조적 코드 재설계 (기능 변경 없음)

test / chore

테스트 코드 추가 및 수정 / 빌드 패키지 매니저 설정 변경, 단순 에셋 변경 등 부차적 작업

Issue 개념 및 연계

■ 개념 및 역할

프로젝트 관리의 최소 단위이자, 버그나 신규 기능 개발 논의를 시작하는 출발점

■ 티켓 시스템 연계

현대 협업 도구(Jira, Trello, GitHub Projects 등)의 모든 백로그(Backlog)와 작업 티켓(Ticket)의 원형 모델임

Issue Template 작성

■ 목적

제보자가 불완전한 정보로 이슈를 등록해 소통 비용이 낭비되는 문제를 차단함

■ 핵심 구조

1. 버그 리포트

상세 현상, 재현 경로(Steps), 기대 동작(Expected), 실제 동작
(Actual), OS/브라우저 환경

2. 기능 제안

제안 배경, 구체적 구현 내용, 대안이나 참고 스크린샷

GitHub Flow 핵심 4단계

- **정의:** GitHub가 권장하는 단순하고 기민한(Agile) 브랜치 기반 배포 모델

- **핵심 4단계 워크플로우:**

1. **브랜치 분기:**

언제나 동작하는 안전한 main 브랜치에서 기능별 신규 브랜치 분기 (main - feature/기능명)

2. **커밋 & 푸시:**

작업 단위를 커밋하고 수시로 원격 저장소에 푸시하여 분실 방지 및 진행 상황 공유

3. **Pull Request 생성:**

코드 작성이 완료되거나 피드백이 필요한 시점에 PR을 열고 리뷰어 지정

4. **리뷰, 병합 & 배포:**

동료들의 피드백을 수용하여 코드 병합 후 즉시 상용(Production) 배포

기타 브랜치 관리 전략 비교

Git Flow

대규모 배포 주기를 가진 전통적 제품 최적화 모델. 브랜치를 5종류로 나눠 정밀 통제함

master

develop

feature

release

hotfix

GitLab Flow

배포 환경 단위(Staging, Production 등)의 브랜치를 두어 점진적 배포에 강점을 지님

실무 브랜치 세분화 팁

개인 개발자 로컬 브랜치 → 팀 공유 파트 브랜치 → 피쳐 단위 브랜치로 나눈 뒤 최종 main에 귀속시켜 대형 충돌 방지

Pull Request (PR) 개념 및 범주

■ 개념

수정한 내역을 대상 브랜치에 병합(Merge)해 줄 것을 프로젝트 동료들에게 공식 요청하는 협업 도구

■ PR의 범주

내부 브랜치 PR

동일한 레포지토리 내에서 두 브랜치 간 병합 요청

포크 기반 PR

외부 작업자가 복제한 개인 저장소에서 원본 메인
저장소로의 외부 병합 요청

PR 본문 작성 3대 골자

1. 요약 (Summary)

어떤 문제를 해결하기 위해 코드를 수정했는지 의도 기술

2. 작업 내역 (Changes)

변경 사항 리스트 (체크리스트 포맷 권장)

3. 이슈 연동 (Issue Linking) `Closes #이슈번호`

본문에 형태로 작성 시, PR 병합과 동시에 이슈가 자동으로 닫히도록 자동화 설정

Forking Workflow 배경

⚠ Collaborator 쓰기 권한 부여 방식의 한계

다수의 오픈소스 기여자나 외부 협력 인력에게 원본 저장소의 직접 쓰기(Write) 권한을 일일이 제공하는 것은 **심각한 보안 리스크**이자 실수로 코드를 날려 먹을 **위험 요소**가 됨.

→ 이를 해결하기 위해 **Forking Workflow** 아키텍처를 도입하여 원본 저장소를 보호함.

Forking Workflow 동작 원리

1 원본 저장소(Upstream)를 나의 개인 GitHub 계정으로 **복제(Fork)**하여 나만의 원격 저장소(Origin)를 확보함

2 내 개인 원격 저장소(Origin)를 로컬로 클론하여 작업을 진행하고 푸시함

3 개인 저장소의 특정 브랜치에서 원본 저장소(Upstream)의 메인 브랜치를 대상으로 **Pull Request**를 발행함

4 원본 저장소 관리자가 코드를 정밀 검토하고 병합을 수락함

코드리뷰 자동화 (Gemini Code Assist)

■ 역할

PR이 생성되는 순간 AI 기반 어시스턴트를 자동 트리거하여 소스코드의 취약점, 안티패턴, 리팩토링 제안을 1차적으로 수행함

■ 장점

리뷰어 피로 단축

단순 오타, 컨벤션 위반, Null Check 누락 등 기계적 검증을 AI가 선제 필터링하여 리뷰어는 비즈니스 로직에 집중함

정적 분석 고도화

지속적인 통합(CI) 파이프라인과 결합하여 PR이 열릴 때마다 즉각적인 코드 진단 피드백 환류 가능